

Федеральное государственное автономное образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

Лабораторная работа №6

«Численное решение обыкновенных
дифференциальных уравнений»

по курсу «Вычислительная математика»

Вариант 19

Выполнил:

Якименко Владислав Игоревич, Р3213

Проверила:

Малышева Татьяна Алексеевна

Содержание

1	Цель работы и постановка задачи	1
1.1	Задача Коши	1
1.2	Вариант задания	1
1.3	Уравнения, предлагаемые программой	1
2	Описание алгоритма решения задачи	2
2.1	Метод конечных разностей	2
2.2	Общая схема работы программы	2
3	Рабочие формулы используемых методов	4
3.1	Метод Эйлера (метод 1)	4
3.2	Метод Рунге–Кутты 4-го порядка (метод 3)	4
3.3	Метод Адамса, схема предиктор-корректор (метод 4)	4
3.4	Оценка погрешности одношаговых методов: правило Рунге	5
3.5	Оценка погрешности многошагового метода	5
4	Программная реализация	6
4.1	Структура проекта	6
4.2	Ввод исходных данных	6
4.3	Листинг программы	7
5	Результаты выполнения программы	14
5.1	Задача варианта	14
5.2	Правило Рунге	15
5.3	Порядок сходимости	15
5.4	Другие наборы исходных данных	16
5.5	Проверка программы	16
6	Графики точного и приближённого решений	18
7	Скриншоты результатов работы программы	21
8	Вывод	25

1 Цель работы и постановка задачи

Цель работы: решить задачу Коши для обыкновенного дифференциального уравнения первого порядка численными методами, сравнить одношаговые и многошаговые схемы и оценить погрешность полученных решений.

1.1 Задача Коши

Требуется найти функцию $y = y(x)$, удовлетворяющую уравнению первого порядка, разрешённому относительно производной,

$$y' = f(x, y), \quad x \in [x_0, x_n],$$

и начальному условию

$$y(x_0) = y_0.$$

Решить задачу численно — значит для последовательности узлов $x_0 < x_1 < \dots < x_n$ с постоянным шагом $h = x_{i+1} - x_i$ приближённо вычислить значения $y_1, \dots, y_n$ сеточной функции, приближающей точное решение в этих узлах.

Достаточные условия существования и единственности даёт теорема Коши: если $f(x, y)$ непрерывна в замкнутой области $\bar{D} = \{(x, y) : |x - x_0| \leq a, |y - y_0| \leq b\}$, решение существует; если, кроме того, f удовлетворяет условию Липшица по y

$$|f(x, y_1) - f(x, y_2)| \leq k|y_1 - y_2|, \quad k = \text{const} > 0,$$

то решение единственно. Геометрически задача Коши — это поиск интегральной кривой, проходящей через заданную начальную точку $M_0(x_0, y_0)$.

1.2 Вариант задания

Номер варианта — 19. По таблице 1 задания варианту соответствует набор методов **1, 3, 4**:

- одношаговые: **метод Эйлера** (метод 1) и **метод Рунге–Кутты 4-го порядка** (метод 3);
- многошаговый: **метод Адамса** в схеме предиктор-корректор (метод 4).

Точность одношаговых методов оценивается по **правилу Рунге**, точность многошагового — по точному решению: $\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|$.

1.3 Уравнения, предлагаемые программой

Задание требует предложить пользователю не менее трёх уравнений. Программа предлагает пять; для каждого известно аналитическое решение задачи Коши, поэтому погрешность можно измерить в каждом узле.

Таблица 1: Уравнения и точные решения задачи Коши

Идентификатор	$y' = f(x, y)$	Точное решение
linear	y	$y = y_0 e^{x-x_0}$
sum	$x + y$	$y = (y_0 + x_0 + 1)e^{x-x_0} - x - 1$
square	$y - x^2 + 1$	$y = (x + 1)^2 + Ce^x, C = (y_0 - (x_0 + 1)^2)e^{-x_0}$
bernoulli	$y + (1 + x)y^2$	$y = \frac{1}{Ce^{-x} - x}, C = e^{x_0} \left(\frac{1}{y_0} + x_0 \right)$
gauss	$-2xy$	$y = y_0 e^{x_0^2 - x^2}$

Уравнение `bernoulli` совпадает с примером 1 лекции: при $y(1) = -1$ постоянная $C = e \left(\frac{1}{-1} + 1 \right) = 0$, и точное решение принимает вид $y = -\frac{1}{x}$. Это позволяет сверить работу программы с таблицами, приведёнными в лекции, поэтому именно эта задача выбрана в качестве основного демонстрационного набора:

$$y' = y + (1 + x)y^2, \quad y(1) = -1, \quad x \in [1; 1.5], \quad h = 0.1, \quad \varepsilon = 10^{-6}.$$

2 Описание алгоритма решения задачи

2.1 Метод конечных разностей

Все реализованные методы основаны на методе конечных разностей. Область непрерывного изменения аргумента — отрезок $[x_0, x_n]$ — заменяется дискретным множеством узлов $x_i = x_0 + ih$, образующих разностную сетку. Искомая функция непрерывного аргумента заменяется сеточной функцией y_i , а дифференциальное уравнение — разностным уравнением относительно y_i ; входящая в уравнение производная заменяется конечно-разностным отношением. Такая замена называется разностной аппроксимацией на сетке.

2.2 Общая схема работы программы

1. Пользователь выбирает уравнение $y' = f(x, y)$ из предложенного списка.
2. С клавиатуры (либо из файла) вводятся x_0 , $y_0 = y(x_0)$, правая граница x_n , шаг h и точность ε .
3. Входные данные проверяются: $x_n > x_0$, $h > 0$, $h \leq x_n - x_0$, $0 < \varepsilon < 1$, число узлов не превышает предела, правая часть определена в начальной точке.
4. Строится равномерная сетка, содержащая обе границы. Если длина интервала не делится на заданный шаг, фактический шаг немного уменьшается. Вычисляется точное решение $y_{i \text{ точн}}$ в узлах.

5. Задача решается тремя методами варианта. Метод Адамса перед началом счёта получает y_1, y_2, y_3 методом Рунге–Кутта.
6. Составляется общая таблица приближённых значений и погрешностей.
7. Для одношаговых методов выполняется контроль точности по правилу Рунге: задача решается с шагами h и $h/2$, вычисляется R ; пока $R > \varepsilon$, шаг делится пополам. Итоговым становится решение на принятой уточнённой сетке.
8. Для метода Адамса вычисляется $\max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|$; пока максимум больше ε , шаг делится пополам и расчёт повторяется.
9. Строятся графики точного и приближённых решений (разными цветами) и график погрешности; результаты выводятся в консоль, в файл и в браузер.

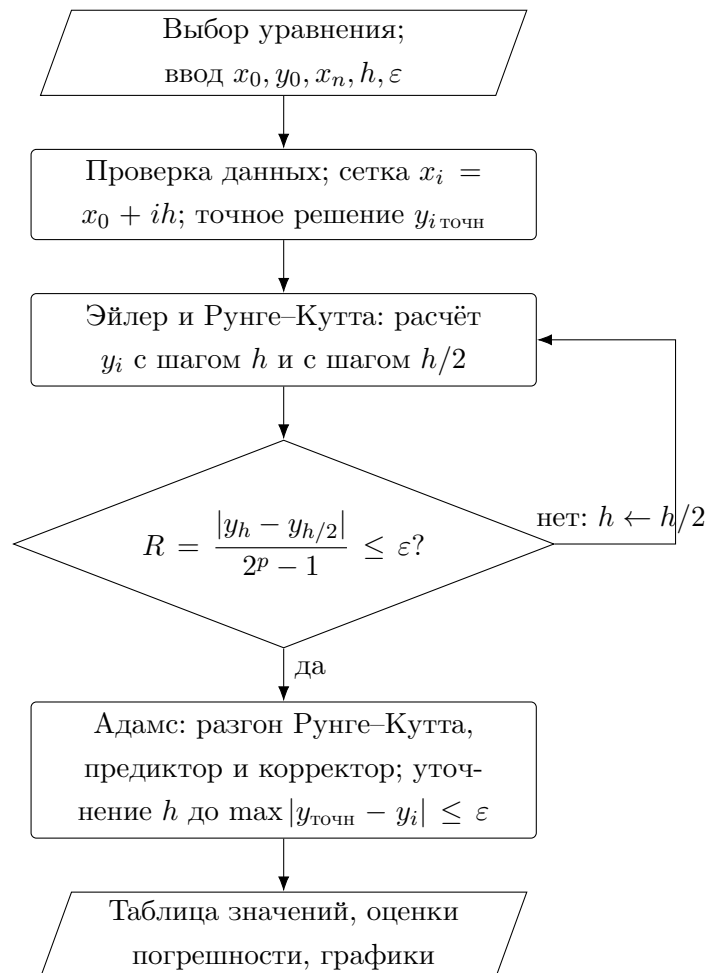


Рис. 1: Блок-схема алгоритма программы

3 Рабочие формулы используемых методов

3.1 Метод Эйлера (метод 1)

Метод основан на разложении решения в ряд Тейлора в окрестности узла x_i с отбрасыванием членов, содержащих производные второго и более высоких порядков:

$$Y(x_i + h) = Y(x_i) + Y'(x_i)h + O(h^2), \quad Y'(x_i) = f(x_i, y_i) \approx \frac{y_{i+1} - y_i}{h}.$$

Отсюда получается расчётная формула

$$y_{i+1} = y_i + hf(x_i, y_i), \quad i = 0, 1, \dots, n-1, \quad (1)$$

$$y_0 = Y_0. \quad (2)$$

Геометрически на каждом шаге приращение решения заменяется приращением ординаты касательной, поэтому после первого шага счёт продолжается уже по другой интегральной кривой. Метод одношаговый, порядок точности $p = 1$, суммарная погрешность $\delta_n = O(h)$.

3.2 Метод Рунге–Кутты 4-го порядка (метод 3)

Идея методов Рунге–Кутты: не использовать частные производные $f(x, y)$, а вычислять саму правую часть в нескольких точках внутри шага. Для наиболее распространённого метода 4-го порядка:

$$y_{i+1} = y_i + \frac{1}{6} (k_1 + 2k_2 + 2k_3 + k_4), \quad (3)$$

$$\begin{aligned} k_1 &= hf(x_i, y_i), & k_2 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_1}{2}\right), \\ k_3 &= hf\left(x_i + \frac{h}{2}, y_i + \frac{k_2}{2}\right), & k_4 &= hf(x_i + h, y_i + k_3). \end{aligned} \quad (4)$$

Метод требует четырёхкратного вычисления правой части на каждом шаге, но имеет суммарную погрешность $\delta_n = O(h^4)$, что позволяет считать с существенно большим шагом, чем метод Эйлера. Метод одношаговый: значение y_{i+1} строится только по y_i , поэтому шаг можно менять в любой момент.

3.3 Метод Адамса, схема предиктор-корректор (метод 4)

Многошаговые методы получают интегрированием уравнения на каждом отрезке. Правую часть заменяют интерполяционным многочленом P_{k-1} , построенным по значениям $f_{i-k+1}, \dots, f_i$:

$$y_{i+1} = y_i + \int_{x_i}^{x_{i+1}} P_{k-1}(x) dx.$$

При $k = 4$ и постоянном шаге многочлен Ньютона строится по конечным разностям правой части

$$\Delta f_i = f_i - f_{i-1}, \quad \Delta^2 f_i = f_i - 2f_{i-1} + f_{i-2}, \quad \Delta^3 f_i = f_i - 3f_{i-1} + 3f_{i-2} - f_{i-3},$$

и разностная схема метода Адамса 4-го порядка принимает вид

$$y_{i+1} = y_i + h \left(f_i + \frac{1}{2} \Delta f_i + \frac{5}{12} \Delta^2 f_i + \frac{3}{8} \Delta^3 f_i \right). \quad (5)$$

Раскрыв разности, получаем явную формулу с числовыми коэффициентами. Именно она используется в программе как **этап предиктора**:

$$y_{i+1}^{\text{прогн}} = y_i + \frac{h}{24} (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3}). \quad (6)$$

Этап корректора использует неявную формулу, в правую часть которой входит уже уточняемое значение через $f_{i+1} = f(x_{i+1}, y_{i+1})$:

$$y_{i+1}^{(s+1)} = y_i + \frac{h}{24} \left(9f(x_{i+1}, y_{i+1}^{(s)}) + 19f_i - 5f_{i-1} + f_{i-2} \right). \quad (7)$$

Итерации корректора продолжаются, пока $|y_{i+1}^{(s+1)} - y_{i+1}^{(s)}| > \varepsilon$; при выполнении обратного неравенства $|y_{i+1}^{(s+1)} - y_{i+1}^{(s)}| \leq \varepsilon$ выполняется переход к следующему узлу. Начальным приближением $y_{i+1}^{(0)}$ служит предиктор (6).

Формулы (6) и (7) требуют значений в четырёх предыдущих узлах, поэтому счёт может быть начат только с y_4 . Значения y_1, y_2, y_3 получены методом Рунге–Кутты 4-го порядка, y_0 задано начальным условием. Суммарная погрешность метода $\delta_n = O(h^4)$; при этом на каждом шаге правая часть вычисляется всего один-два раза против четырёх у метода Рунге–Кутты.

Замечание. В лекции формула (17) записана в виде $y_{i+1} = y_i + hf_i + \frac{h^2}{2} \Delta f_i + \frac{5h^3}{12} \Delta^2 f_i + \frac{3h^4}{8} \Delta^3 f_i$. Конечные разности $\Delta^k f_i$ уже безразмерны относительно h , поэтому множителем при всех слагаемых должно быть h в первой степени — как в (5). Именно в такой записи (5) раскрывается ровно в (6), что подтверждено автоматическим тестом.

3.4 Оценка погрешности одношаговых методов: правило Рунге

Для метода порядка p решение вычисляется дважды — с шагом h и с шагом $h/2$, — после чего погрешность оценивается как

$$R = \frac{y_h - y_{h/2}}{2^p - 1}, \quad (8)$$

где y_h и $y_{h/2}$ — решения в одном и том же узле. В программе контроль выполняется на всём интервале: R вычисляется во всех общих узлах двух сеток и берётся максимум по модулю. Если $|R| \leq \varepsilon$, точность считается достигнутой; иначе шаг делится пополам и расчёт повторяется. Для метода Эйлера $p = 1$, для метода Рунге–Кутты $p = 4$.

3.5 Оценка погрешности многошагового метода

Для метода Адамса, как требует задание, используется точное решение:

$$\varepsilon = \max_{0 \leq i \leq n} |y_{i \text{ точн}} - y_i|. \quad (9)$$

Такая оценка применима именно потому, что все предлагаемые программой уравнения имеют аналитическое решение. Она же вычисляется и для одношаговых методов — это позволяет сравнить фактическую погрешность с оценкой (8). Если для метода Адамса максимум превышает заданное ε , программа делит шаг пополам и повторяет расчёт.

4 Программная реализация

4.1 Структура проекта

Программа написана на JavaScript для Node.js 20+, сторонних зависимостей нет. Численные методы вынесены в отдельный модуль `src/ode.js`: каждый метод — это самостоятельная функция с одинаковой сигнатурой (f, x_0, y_0, x_n, h) , а список `METHODS` связывает метод с его порядком точности и способом оценки погрешности.

Таблица 2: Состав проекта

Путь	Содержимое
<code>src/ode.js</code>	Уравнения, проверка данных, все методы, оценка погрешности
<code>src/io.js</code>	Чтение задачи из файла и форматирование таблиц
<code>src/index.js</code>	Консольное меню, ввод с клавиатуры, параметры запуска
<code>src/plotting-core.js</code>	Построение SVG-графиков решений и погрешности
<code>src/gui-server.js</code>	Локальный сервер веб-интерфейса
<code>src/report.js</code>	Пересчёт результатов, текстовых файлов и графиков
<code>src/ode.test.js</code>	31 автоматический тест
<code>web/</code>	Интерфейс браузера
<code>examples/</code>	4 корректных набора, текстовый набор, 4 ошибочных
<code>output/, docs/</code>	Результаты запусков, графики, снимки экрана

4.2 Ввод исходных данных

Данные задаются тремя способами:

- с клавиатуры — интерактивное меню выбирает уравнение и запрашивает $x_0, y_0, x_n, h, \varepsilon$ (десятичная запятая допускается);
- из файла — JSON либо текст вида «имя = значение»;
- параметрами командной строки `--equation, --x0, --y0, --xn, --h, --eps`.

Тот же расчётный модуль используется веб-интерфейсом, поэтому консоль и браузер дают одинаковые числа.

4.3 Листинг программы

Ниже приведён расчётный модуль `src/ode.js` целиком: описание уравнений с точными решениями, проверка входных данных, метод Эйлера, метод Рунге–Кутты 4-го порядка, метод Адамса в схеме предиктор-корректор, правило Рунге и оценка по точному решению.

```
1 import { parseNumber } from "./utils.js";
2
3 // Каждое уравнение задаётся правой частью  $f(x, y)$  и точным решением задачи Коши.
4 // exact(x0, y0) возвращает функцию  $y(x)$  либо null, если условие недопустимо.
5 export const EQUATIONS = [
6   {
7     id: "linear",
8     label: " $y' = y$ ",
9     solutionLabel: " $y = y_0 \cdot e^{-(x-x_0)}$ ",
10    f: (x, y) => y,
11    exact: (x0, y0) => x => y0 * Math.exp(x - x0),
12  },
13  {
14    id: "sum",
15    label: " $y' = x + y$ ",
16    solutionLabel: " $y = (y_0 + x_0 + 1) \cdot e^{-(x-x_0)} - x - 1$ ",
17    f: (x, y) => x + y,
18    exact: (x0, y0) => x => (y0 + x0 + 1) * Math.exp(x - x0) - x - 1,
19  },
20  {
21    id: "square",
22    label: " $y' = y - x^2 + 1$ ",
23    solutionLabel: " $y = (x+1)^2 + C \cdot e^{-x}$ ,  $C = (y_0 - (x_0+1)^2) \cdot e^{-(x_0)}$ ",
24    f: (x, y) => y - x * x + 1,
25    exact: (x0, y0) => {
26      const c = (y0 - (x0 + 1) ** 2) * Math.exp(-x0);
27      return x => (x + 1) ** 2 + c * Math.exp(x);
28    },
29  },
30  {
31    id: "bernoulli",
32    label: " $y' = y + (1+x) \cdot y^2$ ",
33    solutionLabel: " $y = 1/(C \cdot e^{-x} - x)$ ,  $C = e^{-(x_0)} \cdot (1/y_0 + x_0)$ ",
34    note: "Пример 1 из лекции:  $y(1) = -1$  на  $[1; 1.5]$  даёт  $y = -1/x$ .",
35    f: (x, y) => y + (1 + x) * y * y,
36    exact: (x0, y0) => {
37      if (y0 === 0) return x => 0;
38      const c = Math.exp(x0) * (1 / y0 + x0);
39      return x => {
40        const denominator = c * Math.exp(-x) - x;
41        return denominator === 0 ? Number.NaN : 1 / denominator;
42      };
43    },
44    singularity: (x0, y0, xn) => {
45      if (y0 === 0) return null;
46      const c = Math.exp(x0) * (1 / y0 + x0);
47      const denominator = x => c * Math.exp(-x) - x;
48      const points = [x0, xn];
49      if (c < 0) {
50        const critical = Math.log(-c);
51        if (critical > x0 && critical < xn) points.push(critical);
52      }
53      points.sort((a, b) => a - b);
54      for (let i = 0; i < points.length; i++) {
55        const value = denominator(points[i]);
56        if (Math.abs(value) <= 1e-12 * Math.max(1, Math.abs(points[i]))) return points[i];
57        if (i === 0) continue;
58        let left = points[i - 1], right = points[i];
59        let leftValue = denominator(left), rightValue = value;
60        if (Math.sign(leftValue) === Math.sign(rightValue)) continue;
```

```

61     for (let iteration = 0; iteration < 80; iteration++) {
62         const middle = (left + right) / 2;
63         const middleValue = denominator(middle);
64         if (Math.sign(middleValue) === Math.sign(leftValue)) {
65             left = middle; leftValue = middleValue;
66         } else {
67             right = middle; rightValue = middleValue;
68         }
69     }
70     return (left + right) / 2;
71 }
72 return null;
73 },
74 },
75 {
76     id: "gauss",
77     label: "y' = -2·x·y",
78     solutionLabel: "y = ,y·e^(,x²-x²)",
79     f: (x, y) => -2 * x * y,
80     exact: (x0, y0) => x => y0 * Math.exp(x0 * x0 - x * x),
81 },
82 ];
83
84 export const MAX_NODES = 200000;
85 export const MAX_REFINEMENTS = 20;
86
87 // Принимает идентификатор либо само уравнение, поэтому validateProblem идемпотентна.
88 export function findEquation(id) {
89     if (id && typeof id === "object") id = id.id;
90     const equation = EQUATIONS.find(item => item.id === id);
91     if (!equation) throw new ErrorНеизвестное(`уравнение «${id}». Доступны: ${EQUATIONS.map(e => e.id).join(", ")}
92     });
93     return equation;
94 }
95
96 export function validateProblem(input) {
97     if (!input || typeof input !== "object") throw new Error("Требуется описание задачи Коши.");
98     const equation = findEquation(input.equation ?? input.equationId);
99     const x0 = parseNumber(input.x0, ",x");
100    const y0 = parseNumber(input.y0, ",y");
101    const xn = parseNumber(input.xn, "xn");
102    const requestedH = parseNumber(input.requestedH ?? input.h, "Шаг h");
103    const epsilon = parseNumber(input.epsilon ?? 1e-6, "Точность ε");
104    if (xn <= x0) throw new Error("Правая граница xn должна быть больше ,x.");
105    if (requestedH <= 0) throw new Error("Шаг h должен быть положительным.");
106    if (requestedH > xn - x0) throw new Error("Шаг h не должен превышать длину интервала.");
107    if (epsilon <= 0) throw new Error("Точность ε должна быть положительной.");
108    if (epsilon >= 1) throw new Error("Точность ε должна быть меньше единицы.");
109    const count = gridStepCount(x0, xn, requestedH);
110    if (count + 1 > MAX_NODES) throw new ErrorСлишком(`мелкий шаг: узлов больше ${MAX_NODES}.`);
111    const h = (xn - x0) / count;
112    gridNodes(x0, xn, h);
113    const singularity = equation.singularity?.(x0, y0, xn);
114    if (Number.isFinite(singularity)) {
115        throw new ErrorТочное(`решение имеет полюс внутри интервала около x = ${singularity.toPrecision(8)}.
116        Измените начальные условия или интервал.`);
117    }
118    const problem = { equation, x0, y0, xn, h, requestedH, epsilon };
119    const first = equation.f(x0, y0);
120    if (!Number.isFinite(first)) throw new Error("Правая часть не определена в начальной точке.");
121    return problem;
122 }
123
124 function gridStepCount(x0, xn, h) {
125     const ratio = (xn - x0) / h;
126     const nearest = Math.round(ratio);
127     const almostInteger = Math.abs(ratio - nearest) <= 1e-12 * Math.max(1, Math.abs(ratio));
128     return Math.max(1, almostInteger ? nearest : Math.ceil(ratio));
129 }

```

```

128
129 // Сетка всегда равномерная, содержит обе границы, а её фактический шаг не больше заданного.
130 export function gridNodes(x0, xn, h) {
131     const count = gridStepCount(x0, xn, h);
132     const step = (xn - x0) / count;
133     const nodes = Array.from({ length: count + 1 }, (_, i) => i === count ? xn : x0 + i * step);
134     for (let i = 1; i < nodes.length; i++) {
135         if (!(nodes[i] > nodes[i - 1])) throw new Error("Шаг неразличим в машинной арифметике при заданном .x.");
136     }
137     return nodes;
138 }
139
140 function guard(value, method, x) {
141     if (!Number.isFinite(value)) {
142         throw new Error(`${method}: решение ушло в бесконечность около x = ${x.toPrecision(6)}. Уменьшите шаг или интервал.`);
143     }
144     return value;
145 }
146
147 // ----- Одношаговые методы -----
148
149 // Метод Эйлера:  $y_{i+1} = y_i + h \cdot f(x_i, y_i)$ . Порядок точности  $p = 1$ .
150 export function euler(f, x0, y0, xn, h) {
151     const nodes = gridNodes(x0, xn, h);
152     const values = [y0];
153     for (let i = 1; i < nodes.length; i++) {
154         const step = nodes[i] - nodes[i - 1];
155         values.push(guard(values[i - 1] + step * f(nodes[i - 1], values[i - 1]), "Метод Эйлера", nodes[i]));
156     }
157     return { nodes, values };
158 }
159
160 // Усовершенствованный метод Эйлера метод( Эйлера с пересчётом),  $p = 2$ .
161 // Реализован для сравнения; в вариант 19 не входит.
162 export function improvedEuler(f, x0, y0, xn, h) {
163     const nodes = gridNodes(x0, xn, h);
164     const values = [y0];
165     for (let i = 1; i < nodes.length; i++) {
166         const step = nodes[i] - nodes[i - 1];
167         const slope = f(nodes[i - 1], values[i - 1]);
168         const predictor = values[i - 1] + step * slope;
169         const next = values[i - 1] + step / 2 * (slope + f(nodes[i], predictor));
170         values.push(guard(next, "Усовершенствованный метод Эйлера", nodes[i]));
171     }
172     return { nodes, values };
173 }
174
175 // Метод РунгеКутты- го4- порядка,  $p = 4$ .
176 export function rungeKutta4(f, x0, y0, xn, h) {
177     const nodes = gridNodes(x0, xn, h);
178     const values = [y0];
179     for (let i = 1; i < nodes.length; i++) {
180         const step = nodes[i] - nodes[i - 1];
181         const x = nodes[i - 1], y = values[i - 1];
182         const k1 = step * f(x, y);
183         const k2 = step * f(x + step / 2, y + k1 / 2);
184         const k3 = step * f(x + step / 2, y + k2 / 2);
185         const k4 = step * f(x + step, y + k3);
186         values.push(guard(y + (k1 + 2 * k2 + 2 * k3 + k4) / 6, "Метод РунгеКутты-", nodes[i]));
187     }
188     return { nodes, values };
189 }
190
191 // ----- Многошаговый метод -----
192
193 // Метод Адамса го4- порядка в схеме предикторкорректор-.
194 // Предиктор АдамсБашфорт(-):  $y_{i+1} = y_i + h/24 \cdot (55f_i - 59f_{i-1} + 37f_{i-2} - 9f_{i-3})$ .
195 // Корректор АдамсМоултон(-):  $y_{i+1} = y_i + h/24 \cdot (9f_{i+1} + 19f_i - 5f_{i-1} + f_{i-2})$ ,

```

```

196 // итерируется до  $|y^{(k+1)} - y^{(k)}| \leq \epsilon$ . Разгон - методом РунгеКутты-.
197 export const ADAMS_MAX_ITERATIONS = 20;
198
199 export function adams(f, x0, y0, xn, h, epsilon = 1e-9) {
200   const nodes = gridNodes(x0, xn, h);
201   if (nodes.length < 5) {
202     throw new Error("Метод Адамса требует не менее четырёх шагов: возьмите  $h \leq (-,xn)/4$ .");
203   }
204   const start = rungeKutta4(f, x0, y0, nodes[3], h);
205   const values = start.values.slice(0, 4);
206   const slopes = values.map((y, i) => f(nodes[i], y));
207   const corrections = [0, 0, 0, 0];
208   for (let i = 3; i < nodes.length - 1; i++) {
209     const step = nodes[i + 1] - nodes[i];
210     let y = values[i] + step / 24 * (55 * slopes[i] - 59 * slopes[i - 1] + 37 * slopes[i - 2] - 9 * slopes[i - 3]);
211     guard(y, "Метод Адамса", nodes[i + 1]);
212     let iterations = 0, delta = Infinity;
213     while (iterations < ADAMS_MAX_ITERATIONS && delta > epsilon) {
214       const corrected = values[i] + step / 24
215         * (9 * f(nodes[i + 1], y) + 19 * slopes[i] - 5 * slopes[i - 1] + slopes[i - 2]);
216       guard(corrected, "Метод Адамса", nodes[i + 1]);
217       delta = Math.abs(corrected - y);
218       y = corrected;
219       iterations++;
220     }
221     if (delta > epsilon) {
222       throw new Error("Метод Адамса: корректор не сошёлся за  $\{ADAMS\_MAX\_ITERATIONS\}$  итераций при  $x = \{nodes[i + 1].toFixed(6)\}$ . Уменьшите шаг  $h$ .");
223     }
224     values.push(y);
225     slopes.push(f(nodes[i + 1], y));
226     corrections.push(iterations);
227   }
228   return { nodes, values, corrections, startedBy: "РунгеКутта-го4- порядка" };
229 }
230
231 // Конечные разности правой части:  $\Delta f_i$ ,  $\Delta^2 f_i$ ,  $\Delta^3 f_i$  - форма (17) из лекции.
232 export function backwardDifferences(slopes, i) {
233   const [f3, f2, f1, f0] = [slopes[i - 3], slopes[i - 2], slopes[i - 1], slopes[i]];
234   return { d1: f0 - f1, d2: f0 - 2 * f1 + f2, d3: f0 - 3 * f1 + 3 * f2 - f3 };
235 }
236
237 // ----- Оценка погрешности -----
238
239 // Правило Рунге:  $R = (y_h - y_{h/2}) / (2^p - 1)$  в общих узлах двух сеток.
240 export function rungeError(coarse, fine, order) {
241   if (fine.nodes.length !== 2 * coarse.nodes.length - 1) {
242     throw new Error("Для правила Рунге требуются вложенные равномерные сетки с отношением шагов 2.");
243   }
244   const denominator = 2 ** order - 1;
245   const tolerance = Math.max(1e-9 * Math.abs(coarse.nodes.at(-1) - coarse.nodes[0]), Number.MIN_VALUE);
246   let max = 0, at = coarse.nodes[0];
247   for (let i = 0; i < coarse.nodes.length; i++) {
248     const j = 2 * i;
249     if (Math.abs(fine.nodes[j] - coarse.nodes[i]) > tolerance) {
250       throw new Error("Узлы сеток  $h$  и  $h/2$  не совпадают.");
251     }
252     const value = Math.abs(coarse.values[i] - fine.values[j]) / denominator;
253     if (value > max) { max = value; at = coarse.nodes[i]; }
254   }
255   return { value: max, at };
256 }
257
258 // Подбор шага по правилу Рунге:  $h$  делится пополам, пока  $R > \epsilon$  на всём интервале.
259 export function refineByRunge(method, order, problem) {
260   const { equation, x0, y0, xn, epsilon } = problem;
261   const steps = [];
262   let h = problem.h;

```

```

263 let coarse;
264 try {
265     coarse = method(equation.f, x0, y0, xn, h);
266 } catch (error) {
267     return { steps, h, converged: false, reason: error.message };
268 }
269 for (let attempt = 0; attempt <= MAX_REFINEMENTS; attempt++) {
270     if (2 * coarse.nodes.length - 1 > MAX_NODES) {
271         return { steps, h, converged: false, solution: coarse, reason: Достигнут` предел ${MAX_NODES} узлов.` };
272     }
273     let fine;
274     try {
275         fine = method(equation.f, x0, y0, xn, h / 2);
276     } catch (error) {
277         return { steps, h, converged: false, solution: coarse, reason: error.message };
278     }
279     const runge = rungeError(coarse, fine, order);
280     const fineH = (xn - x0) / (fine.nodes.length - 1);
281     steps.push({ h: fineH, comparedH: h, nodes: fine.nodes.length, error: runge.value, at: runge.at, ok: runge.value <= epsilon });
282     if (runge.value <= epsilon) return { steps, h: fineH, converged: true, solution: fine, coarse, runge };
283     coarse = fine;
284     h = fineH;
285 }
286 return { steps, h, converged: false, solution: coarse, reason: Точность` не достигнута за ${MAX_REFINEMENTS} делений шага.` };
287 }
288
289 // Погрешность по точному решению:  $\epsilon = \max |y_i \text{ точн} - y_i|$ .
290 export function exactError(nodes, values, exact) {
291     let max = 0, at = nodes[0];
292     for (let i = 0; i < nodes.length; i++) {
293         const reference = exact(nodes[i]);
294         if (!Number.isFinite(reference)) {
295             throw new ErrorТочное(` решение не определено при x = ${nodes[i].toPrecision(8)}.`);
296         }
297         const value = Math.abs(reference - values[i]);
298         if (value > max) { max = value; at = nodes[i]; }
299     }
300     return { value: max, at };
301 }
302
303 export function refineAdams(problem, exact) {
304     const { equation, x0, y0, xn, epsilon } = problem;
305     const steps = [];
306     let h = problem.h;
307     let solution = null;
308     for (let attempt = 0; attempt <= MAX_REFINEMENTS; attempt++) {
309         if (gridStepCount(x0, xn, h) + 1 > MAX_NODES) {
310             return { steps, h, converged: false, solution, reason: Достигнут` предел ${MAX_NODES} узлов.` };
311         }
312         try {
313             solution = adams(equation.f, x0, y0, xn, h, Math.min(epsilon, 1e-10));
314             const error = exactError(solution.nodes, solution.values, exact);
315             const actualH = (xn - x0) / (solution.nodes.length - 1);
316             steps.push({ h: actualH, nodes: solution.nodes.length, error: error.value, at: error.at, ok: error.value <= epsilon });
317             if (error.value <= epsilon) return { steps, h: actualH, converged: true, solution, error };
318             h = actualH / 2;
319         } catch (error) {
320             steps.push({ h, error: Number.POSITIVE_INFINITY, ok: false, reason: error.message });
321             h /= 2;
322         }
323     }
324     return { steps, h, converged: false, solution, reason: Точность` не достигнута за ${MAX_REFINEMENTS} делений шага.` };
325 }
326
327 function sampleAtNodes(solution, targetNodes) {

```

```

328   const sourceCount = solution.nodes.length - 1;
329   const targetCount = targetNodes.length - 1;
330   const ratio = sourceCount / targetCount;
331   if (!Number.isInteger(ratio)) throw new Error("Итоговая сетка метода не вложена в общую сетку.");
332   return targetNodes.map((_, i) => solution.values[i * ratio]);
333 }
334
335 // ----- Сводное решение -----
336
337 export const METHODS = [
338   { id: "euler", label: "Метод Эйлера", short: "Эйлер", kind: "one-step", order: 1, run: euler },
339   { id: "rk4", label: "Метод РунгеКутты-го4- порядка", short: "РунгеКутта-", kind: "one-step", order: 4, run:
     rungeKutta4 },
340   { id: "adams", label: "Метод Адамса предикторкорректор(-)", short: "Адамс", kind: "multistep", order: 4, run:
     adams },
341 ];
342
343 export const VARIANT19_METHODS = ["euler", "rk4", "adams"];
344
345 export function solve(input) {
346   const problem = validateProblem(input);
347   const { equation, x0, y0, xn, h, epsilon } = problem;
348   const exact = equation.exact(x0, y0);
349   const nodes = gridNodes(x0, xn, h);
350   const exactValues = nodes.map(exact);
351   const singular = exactValues.some(value => !Number.isFinite(value));
352   const warnings = [];
353   if (Math.abs(problem.requestedH - problem.h) > 1e-12 * Math.max(1, problem.requestedH)) {
354     warnings.pushДля(` равномерной сетки шаг уменьшен с ${problem.requestedH} до ${problem.h}.`);
355   }
356
357   const results = METHODS.map(method => {
358     const base = { id: method.id, label: method.label, short: method.short, kind: method.kind, order: method.
      order };
359     let initial;
360     try {
361       initial = method.id === "adams"
362         ? method.run(equation.f, x0, y0, xn, h, epsilon)
363         : method.run(equation.f, x0, y0, xn, h);
364     } catch (error) {
365       initial = null;
366     }
367     const baseError = initial ? exactError(initial.nodes, initial.values, exact) : null;
368     const refinement = method.kind === "one-step"
369       ? refineByRunge(method.run, method.order, problem)
370       : refineAdams(problem, exact);
371     if (!refinement.solution) {
372       return { ...base, applicable: false, reason: refinement.reason ?? "Не удалось построить решение." };
373     }
374     const solution = refinement.solution;
375     const error = exactError(solution.nodes, solution.values, exact);
376     if (!refinement.converged) warnings.push(`${method.label}: ${refinement.reason}`);
377     return { ...base, applicable: true, accuracyMet: refinement.converged,
378       nodes: solution.nodes, values: solution.values, commonValues: sampleAtNodes(solution, nodes),
379       h: (xn - x0) / (solution.nodes.length - 1), error,
380       corrections: solution.corrections, base: initial ? { ...initial, error: baseError } : null,
381       runge: method.kind === "one-step" ? refinement : undefined,
382       exactRefinement: method.kind === "multistep" ? refinement : undefined };
383   });
384
385   const spread = (() => {
386     const applicable = results.filter(r => r.applicable);
387     if (applicable.length < 2) return 0;
388     let maximum = 0;
389     for (let i = 0; i < nodes.length; i++) {
390       let minimum = Infinity, maximumInNode = -Infinity, count = 0;
391       for (const result of applicable) {
392         const value = result.commonValues[i];
393         if (!Number.isFinite(value)) continue;

```

```

394     minimum = Math.min(minimum, value);
395     maximumInNode = Math.max(maximumInNode, value);
396     count++;
397 }
398 if (count >= 2) maximum = Math.max(maximum, maximumInNode - minimum);
399 }
400 return maximum;
401 })();
402 return { problem, equation, exact, nodes, exactValues, singular, results, spread, warnings };
403 }

```

5 Результаты выполнения программы

5.1 Задача варианта

Основной набор: $y' = y + (1 + x)y^2$, $y(1) = -1$, $[1; 1,5]$, $h = 0,1$, $\varepsilon = 10^{-6}$. Точное решение $y = -1/x$.

Таблица 3: Итоговые решения в общих узлах исходной сетки

i	x_i	Эйлер	Рунге–Кутта	Адамс	$y_{\text{точн}}$
0	1,0	-1,000000	-1,000000	-1,000000	-1,000000
1	1,1	-0,909090	-0,909091	-0,909091	-0,909091
2	1,2	-0,833333	-0,833334	-0,833333	-0,833333
3	1,3	-0,769230	-0,769231	-0,769230	-0,769231
4	1,4	-0,714285	-0,714286	-0,714285	-0,714286
5	1,5	-0,666666	-0,666667	-0,666666	-0,666667

Таблица 4: Фактическая погрешность и её оценка

Метод	p	Способ оценки	Оценка	$\max y_{\text{точн}} - y_i $
Эйлера	1	правило Рунге	$8,57 \cdot 10^{-7}$ при $h = 6,10 \cdot 10^{-6}$	$8,57 \cdot 10^{-7}$
Рунге–Кутта	4	правило Рунге	$2,34 \cdot 10^{-7}$ при $h = 0,05$	$2,15 \cdot 10^{-7}$
Адамса	4	$\max y_{\text{точн}} - y_i $	$9,80 \cdot 10^{-7}$ при $h = 0,05$	$9,80 \cdot 10^{-7}$

Все три строки таблицы относятся к принятым уточнённым сеткам. Для удобства сравнения итоговые решения показаны в шести общих узлах исходной сетки; максимум погрешности вычислен по всем узлам сетки каждого метода.

Отдельный прямой расчёт на исходном шаге $h = 0,1$ совпадает с таблицами примеров 1 и 3 лекции во всех приведённых там знаках; это совпадение закреплёно автоматическими тестами.

Таблица 5: Сравнение с таблицей примера 1 (метод Эйлера) и примера 3 лекции

x_i	Эйлер, лекция	Эйлер, программа	Рунге–Кутта, лекция	Рунге–Кутта, программа
1,1	-0,900000	-0,900000	-0,909093	-0,909093
1,2	-0,819900	-0,819900	-0,833336	-0,833337
1,3	-0,753998	-0,753998	-0,769234	-0,769234
1,4	-0,698640	-0,698640	-0,714289	-0,714289
1,5	-0,651361	-0,651360	-0,666670	-0,666670

Последний знак местами отличается из-за округления в лекции; программа хранит промежуточные значения в двойной точности.

5.2 Правило Рунге

Для метода Рунге–Кутта заданная точность $\varepsilon = 10^{-6}$ достигается после одной проверки сеток $h = 0,1$ и $h/2 = 0,05$: $R = 2,34 \cdot 10^{-7}$. Методу Эйлера потребовалось 14 сравнений сеток.

Таблица 6: Контроль точности метода Эйлера по правилу Рунге ($p = 1$)

h	Узлов	$R = y_h - y_{h/2} /(2^p - 1)$	$R \leq \varepsilon$
0,05	11	$8,27 \cdot 10^{-3}$	нет
0,025	21	$3,79 \cdot 10^{-3}$	нет
0,0125	41	$1,82 \cdot 10^{-3}$	нет
$6,25 \cdot 10^{-3}$	81	$8,94 \cdot 10^{-4}$	нет
$3,125 \cdot 10^{-3}$	161	$4,43 \cdot 10^{-4}$	нет
$1,5625 \cdot 10^{-3}$	321	$2,20 \cdot 10^{-4}$	нет
$7,8125 \cdot 10^{-4}$	641	$1,10 \cdot 10^{-4}$	нет
$3,9063 \cdot 10^{-4}$	1281	$5,49 \cdot 10^{-5}$	нет
$1,9531 \cdot 10^{-4}$	2561	$2,74 \cdot 10^{-5}$	нет
$9,766 \cdot 10^{-5}$	5121	$1,37 \cdot 10^{-5}$	нет
$4,883 \cdot 10^{-5}$	10241	$6,86 \cdot 10^{-6}$	нет
$2,441 \cdot 10^{-5}$	20481	$3,43 \cdot 10^{-6}$	нет
$1,221 \cdot 10^{-5}$	40961	$1,71 \cdot 10^{-6}$	нет
$6,104 \cdot 10^{-6}$	81921	$8,57 \cdot 10^{-7}$	да

Каждое деление шага уменьшает R примерно вдвое — это прямое подтверждение первого порядка точности метода Эйлера.

Для метода Адамса на исходном шаге получено $\max |y_{\text{точн}} - y_i| = 7,45 \cdot 10^{-6}$. После деления шага пополам при $h = 0,05$ максимум стал равен $9,80 \cdot 10^{-7}$, поэтому требуемая точность достигнута.

5.3 Порядок сходимости

Для той же задачи фактическая погрешность $\max |y_{\text{точн}} - y_i|$ измерена при последовательном дроблении шага.

Таблица 7: Порядок сходимости методов на задаче варианта

h	Эйлер	отношение	Рунге–Кутта	отношение
0,1	$1,5646 \cdot 10^{-2}$	—	$3,7232 \cdot 10^{-6}$	—
0,05	$7,3797 \cdot 10^{-3}$	2,12	$2,1480 \cdot 10^{-7}$	17,33
0,025	$3,5972 \cdot 10^{-3}$	2,05	$1,2883 \cdot 10^{-8}$	16,67
0,0125	$1,7762 \cdot 10^{-3}$	2,03	$7,8814 \cdot 10^{-10}$	16,35

При уменьшении шага вдвое погрешность метода Эйлера уменьшается примерно в $2^1 = 2$ раза, метода Рунге–Кутта — примерно в $2^4 = 16$ раз, что соответствует

заявленным порядкам точности.

5.4 Другие наборы исходных данных

Набор 2: $y' = -2xy$, $y(0) = 1$, $[0; 2]$, $h = 0,25$, $\varepsilon = 10^{-6}$. Точное решение $y = e^{-x^2}$; шаг намеренно взят крупным, чтобы участок перегиба был пройден за несколько узлов.

Таблица 8: Прямой расчёт уравнения $y' = -2xy$ на исходной сетке $h = 0,25$

i	x_i	Эйлер	Рунге–Кутта	Адамс	$y_{\text{точн}}$
0	0,00	1,000000	1,000000	1,000000	1,000000
2	0,50	0,875000	0,778797	0,778797	0,778801
4	1,00	0,410156	0,367935	0,367411	0,367879
6	1,50	0,076904	0,105709	0,105459	0,105399
8	2,00	0,002403	0,018711	0,018527	0,018316

На исходной сетке погрешности равны: Эйлер $9,62 \cdot 10^{-2}$, Рунге–Кутта $4,00 \cdot 10^{-4}$, Адамс $4,69 \cdot 10^{-4}$. Автоматический контроль уменьшает шаги; для Эйлера достижению $\varepsilon = 10^{-6}$ препятствует установленный предел в 200,000 узлов, что программа сообщает предупреждением.

Набор 3: $y' = y - x^2 + 1$, $y(0) = 0,5$, $[0; 2]$, $h = 0,2$ — задан текстовым файлом с десятичной запятой. При прямом расчёте на исходной сетке точное решение $y = (x + 1)^2 - \frac{1}{2}e^x$. В конечной точке $x = 2$: Эйлер 4,865785, Рунге–Кутта 5,305363, Адамс 5,305210, точное 5,305472. Погрешности соответственно $4,40 \cdot 10^{-1}$, $1,09 \cdot 10^{-4}$ и $2,62 \cdot 10^{-4}$.

5.5 Проверка программы

Таблица 9: Наборы данных для проверки

Набор	Содержание проверки	Результат
Вариант 19	$y' = y + (1 + x)y^2$, $y(1) = -1$, [1; 1,5], $h = 0,1$	расчёт выполнен
Уравнение $y' = -2xy$	[0; 2], крупный шаг $h = 0,25$	расчёт выполнен
Текстовый файл	$y' = y - x^2 + 1$, десятичная запятая	файл прочитан
Крупный шаг	$h > (x_n - x_0)/4$	метод Адамса запущен после уменьшения h
Некорректный набор	$h = -0,1$; $x_n < x_0$; $\varepsilon = 0$; неизвестное уравнение	выведена ошибка

Автоматические тесты проекта проверяют совпадение методов Эйлера и Рунге–Кутты с таблицами примеров 1 и 3 лекции, экспериментальные порядки сходимости, разгон метода Адамса, правило Рунге, чтение обоих форматов файлов и обработку некорректных данных.

6 Графики точного и приближённого решений

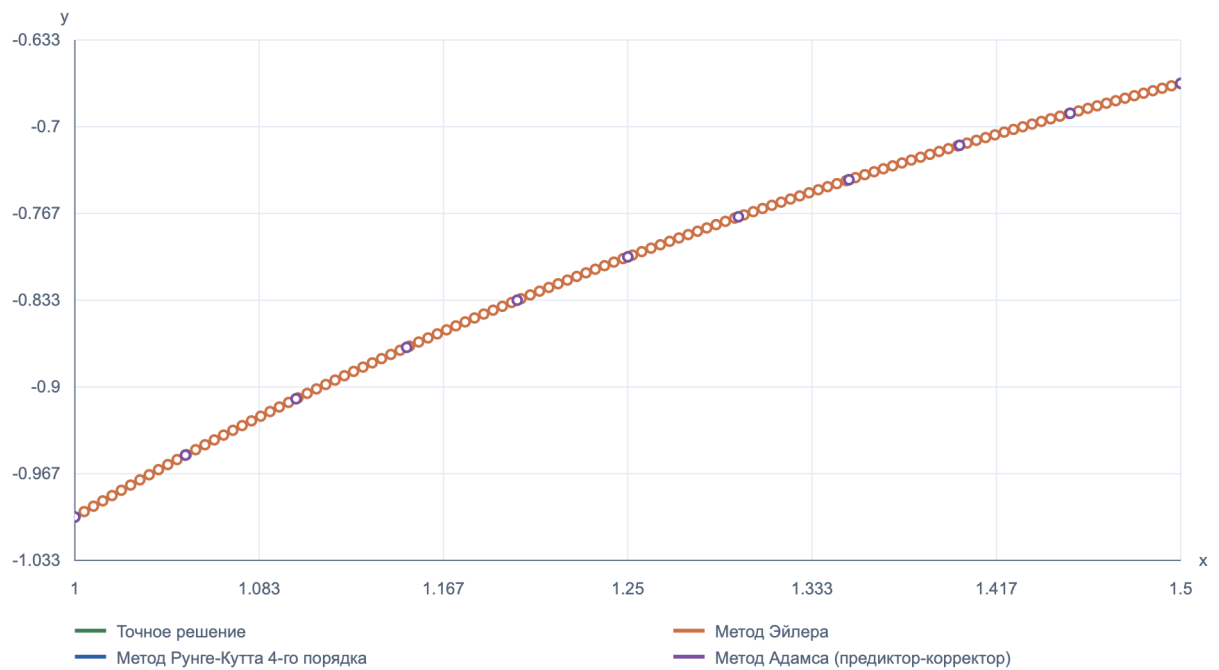


Рис. 2: Задача варианта $y' = y + (1+x)y^2$, $y(1) = -1$: точное решение (зелёная сплошная линия) и принятые приближённые решения (пунктир: оранжевый — Эйлер, синий — Рунге-Кутта, фиолетовый — Адамс)

После контроля точности все три кривые визуально сливаются с точным решением. Методу Эйлера для этого потребовалась значительно более мелкая сетка из-за первого порядка точности.

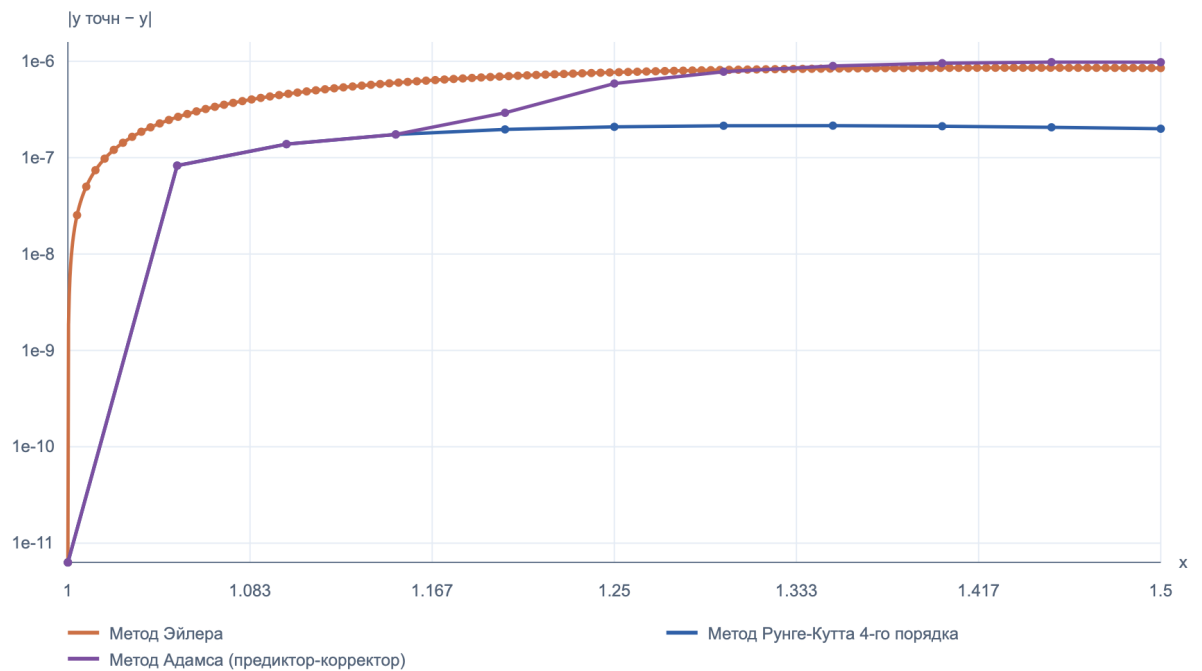


Рис. 3: Погрешность $|y_{\text{точн}} - y_i|$ в узлах, логарифмическая шкала

График погрешности подтверждает, что максимум для каждого принятого решения не превышает 10^{-6} . Разные плотности точек соответствуют индивидуально подобранным шагам методов.

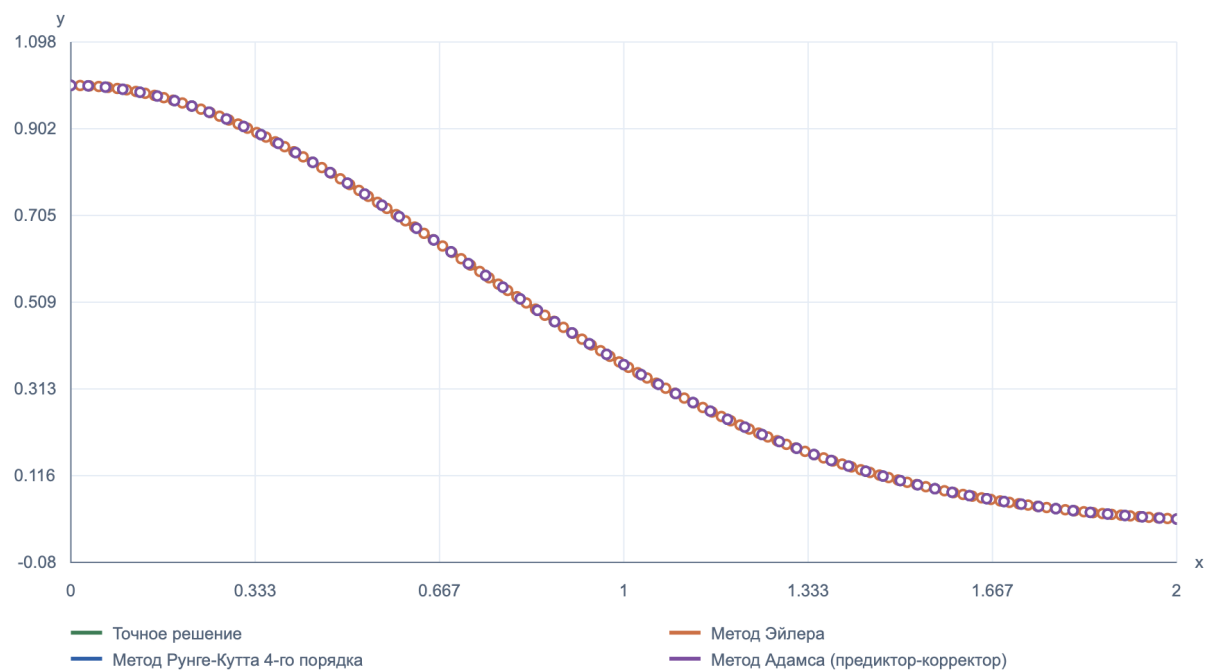


Рис. 4: Уравнение $y' = -2xy$, $y(0) = 1$: решения после контроля точности при исходном шаге $h = 0,25$

7 Скриншоты результатов работы программы

Оформление интерфейса продолжает ЛР №4 и ЛР №5: светлый фон, синие акценты, левая панель ввода и правая область результатов. Ввод x_0 , y_0 , x_n , h , ε и выбор уравнения выполняются в одной форме; таблицы, графики и оценки погрешности обновляются по кнопке «Решить».

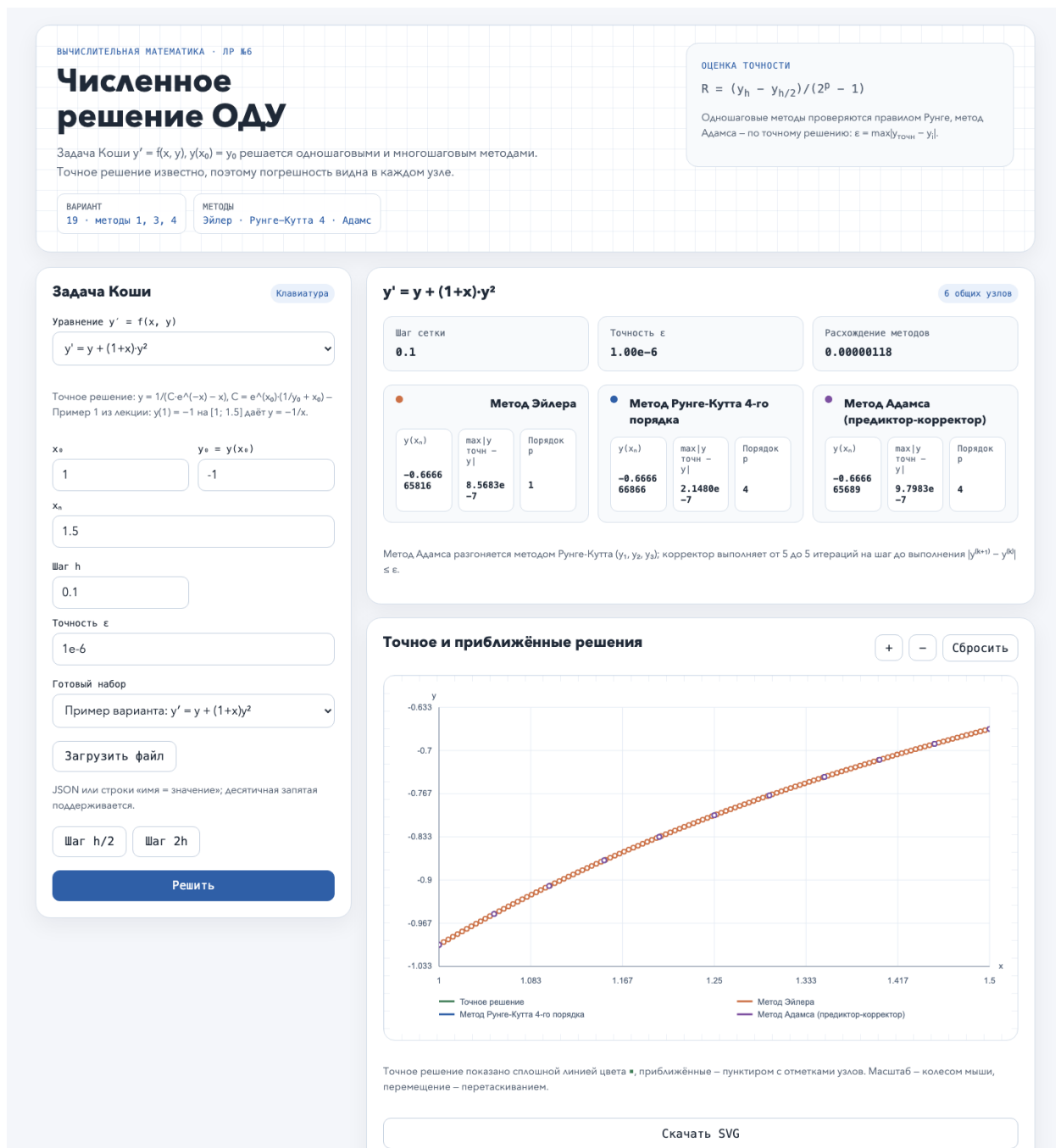


Рис. 5: Набор 1 — задача варианта: параметры, сводка по методам и график точного и приближённых решений

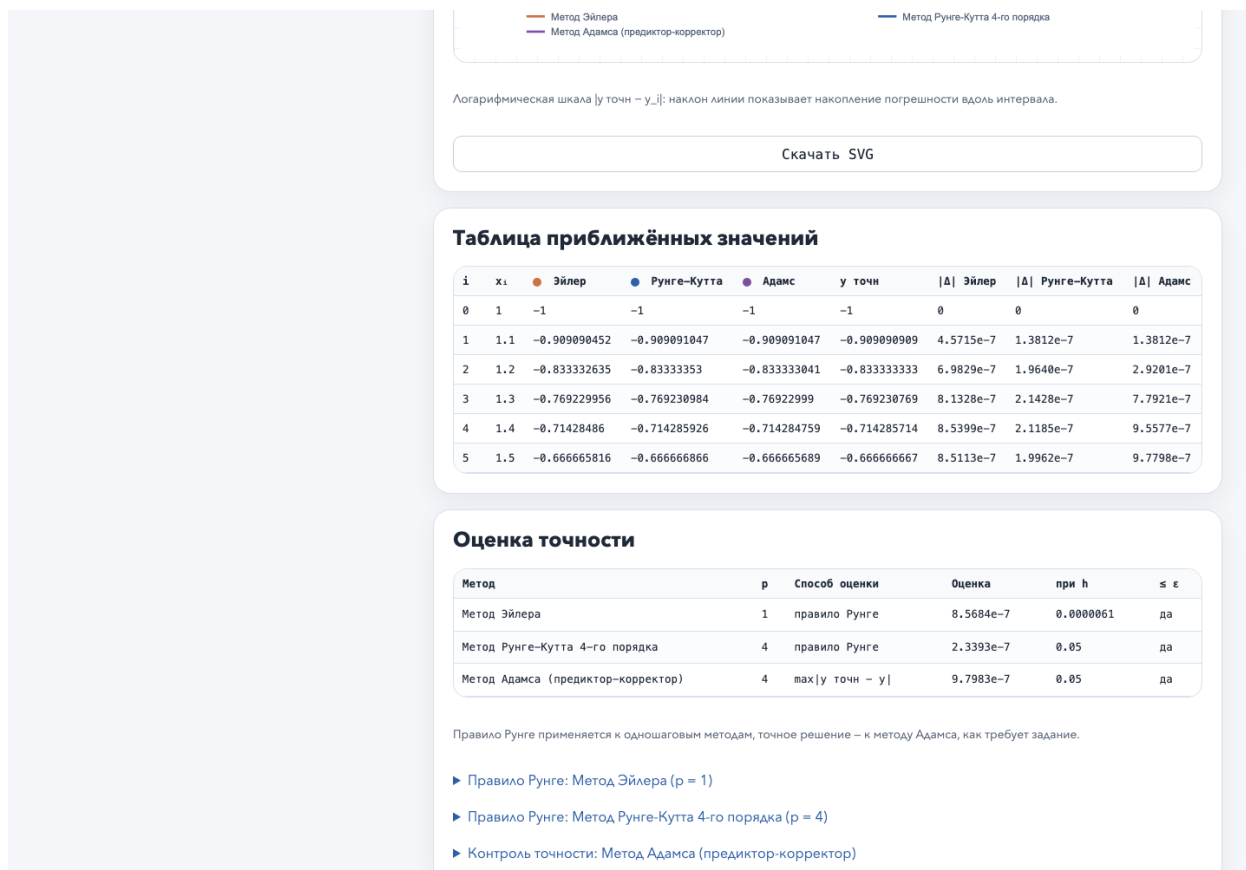


Рис. 6: Тот же запуск: таблица приближённых значений с погрешностью в каждом узле и таблица оценок точности

ВАРИАНТ
19 · методы 1, 3, 4

ОЦЕНКА ТОЧНОСТИ

$$R = (y_h - y_{h/2}) / (2^p - 1)$$

Одношаговые методы проверяются правилом Рунге, метод Адамса – по точному решению: $\varepsilon = \max|y_{\text{точн}} - y_i|$.

Задача Коши

Клавиатура

Уравнение $y' = f(x, y)$

$$y' = y + (1+x) \cdot y^2$$

Точное решение: $y = 1/(C \cdot e^{-x} - x)$, $C = e^{x_0} \cdot (1/y_0 + x_0)$ -
Пример 1 из лекции: $y(1) = -1$ на $[1; 1.5]$ даёт $y = -1/x$.

X₀

1

$$y_0 = y(x_0)$$

-1

 X_n

1.5

War h

-0.1

Точность ε

1e-6

Готовый набор

Пример варианта: $y' = y + (1+x)y^2$

Загрузить файл

JSON или строки «имя = значение»; десятичная запятая поддерживается.

Шаг $h/2$

Шаг 2h

Решить

Шаг h должен быть положительным.

Данные изменены

Нажмите «Решить», чтобы обновить таблицу и графики.

Рис. 7: Некорректные данные: при $h = -0,1$ расчёт не выполняется, выводится диагностическое сообщение

Численное решение ОДУ

Задача Коши $y' = f(x, y)$, $y(x_0) = y_0$ решается одношаговыми и многошаговыми методами. Точное решение известно, поэтому погрешность видна в каждом узле.

ВАРИАНТ
19 · методы 1, 3, 4

МЕТОДЫ
Эйлер · Рунге-Кутта 4 · Адамс

ОЦЕНКА ТОЧНОСТИ

$$R = (y_h - y_{h/2}) / (2^p - 1)$$

Одношаговые методы проверяются правилом Рунге, метод Адамса – по точному решению: $\epsilon = \max |y_{\text{точн}} - y_i|$.

Задача Коши

Клавиатура

Уравнение $y' = f(x, y)$

$$y' = y + (1+x)y^2$$

Точное решение: $y = 1/(C \cdot e^{-(1+x)} - x)$, $C = e^{-(1+x_0)}(1/y_0 + x_0)$ – Пример 1 из лекции: $y(1) = -1$ на $[1; 1.5]$ даёт $y = -1/x$.

x_0

1

$y_0 = y(x_0)$

-1

x_n

1.5

Шаг h

0.1

Точность ϵ

1e-6

Готовый набор

Пример варианта: $y' = y + (1+x)y^2$

Загрузить файл

JSON или строки «имя = значение»; десятичная запятая поддерживается.

Шаг $h/2$

Шаг $2h$

Решить

Рис. 8: Адаптивная форма в узком мобильном окне

8 Вывод

Решена задача Коши $y' = y + (1 + x)y^2$ с начальным условием $y(1) = -1$ на отрезке $[1; 1,5]$ с шагом $h = 0,1$ и точностью $\varepsilon = 10^{-6}$; точное решение $y = -1/x$. После автоматического уточнения шага в точке $x = 1,5$ метод Эйлера дал $-0,666666$, метод Рунге–Кутта 4-го порядка — $-0,666667$, метод Адамса — $-0,666666$ при точном значении $-0,666667$. Максимальная фактическая погрешность равна соответственно $8,57 \cdot 10^{-7}$, $2,15 \cdot 10^{-7}$ и $9,80 \cdot 10^{-7}$, то есть все три метода выполнили условие точности.

При $h = 0,05$ те же погрешности равны $7,38 \cdot 10^{-3}$ и $2,15 \cdot 10^{-7}$, то есть в 2,1 и 17,3 раза меньше, чем при $h = 0,1$, что подтверждает первый и четвёртый порядки методов. По правилу Рунге принято решение Рунге–Кутта при $h = 0,05$ (11 узлов), Эйлера — при $h = 6,10 \cdot 10^{-6}$ (81921 узел). Метод Адамса принят при $h = 0,05$; значения y_1 , y_2 , y_3 вычислены методом Рунге–Кутта. Программа диагностирует неверные параметры, полюс точного решения и переполнение, а для шага, не делящего длину интервала, строит равномерную сетку с обеими границами.